

graças a REP STOSW (16 para setup, 5x76 para toploop, 3 para inter-loop e 5x76 para bottomloop).

As cores do chão e do teto não provêm de arquivos de dados de mapas. O chão tem sempre a mesma cor (0x19) e as cores do teto são definidas diretamente no código do motor gráfico para cada nível.

```
byte vgaTeto [] =
{
    0x1d,0x1d,0x1d,0x1d,0x1d,0x1d,0x1d,0x1d,0x1d,0xbf,0x4e
    ,0x4e,0x4e,0x1d,0x8d,0x4e,0x1d,0x2d,0x1d,0x8d,0x1d,0x1d
    ,0x1d,0x1d,0x1d,0x2d,0xdd,0x1d,0x1d,0x98,

    0x1d,0x9d,0x2d,0xdd,0xdd,0x9d,0x2d,0x4d,0x1d,0xdd,0x7d
    ,0x1d,0x2d,0x2d,0xdd,0xd7,0x1d,0x1d,0x1d,0x2d,0x1d,0x1d
    ,0x1d,0x1d,0xdd,0xdd,0x7d,0xdd,0xdd,0xdd};
```

A seguir, as cores do teto 0x1D, 0xBF, 0x4E e 0x8D.



4.7.5 Resolvendo o problema da CPU

No Capítulo 2 "Hardware", página 27, que descreve as capacidades da CPU, o leitor se depara com um problema: a máquina não consegue realizar operações de ponto flutuante com rapidez suficiente. Isso representa um problema considerável para um motor gráfico 3D, dada toda a trigonometria envolvida. A solução encontrada foi enganar a ULA por meio de uma técnica chamada "aritmética de ponto fixo".

4.7.5.1 Ponto Fixo

O layout normal de um inteiro é o seguinte.

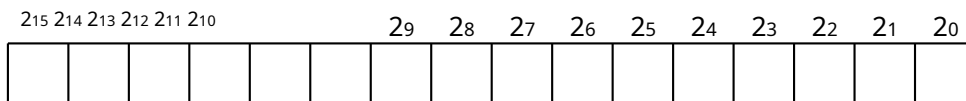


Figura 4.29 Layout de números inteiros.

O valor da sequência de bits *0010001000100010*:

2 ¹⁵	2 ¹⁴	2 ¹³	2 ¹²	2 ¹¹	2 ¹⁰		2 ⁹	2 ⁸	2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰
0	0	1	0	0	0		1	0	0	0	1	0	0	0	1	0

Representa $2^{13} + 2^9 + 2^5 + 2^1 = 8738$.

O ponto fixo permite o controle de frações enquanto ainda utiliza as operações de números inteiros da CPU. A máquina manipula o que deveriam ser números inteiros, mas o programador os vê como um valor contendo uma parte inteira e uma parte fracionária:

2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴	2 ⁻⁵	2 ⁻⁶	2 ⁻⁷	2 ⁻⁸

Figura 4.30: Layout de ponto fixo 8:8 (8 bits para a parte inteira e 8 bits para a parte fracionária).

A mesma sequência de bits *0010001000100010* com diferentes potências de dois...

2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴	2 ⁻⁵	2 ⁻⁶	2 ⁻⁷	2 ⁻⁸
0	0	1	0	0	0	1	0	0	0	1	0	0	0	1	0

Agora representa:

$2^5 + 2^1 = 34$ para a parte inteira.

$2^{-3} + 2^{-7} = 0.1328125$ para a parte fracionária.

= 34.1328125

A beleza do ponto fixo é que a adição e a subtração funcionam exatamente como os números inteiros do ponto de vista das instruções da CPU.

2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	2 ⁻¹	2 ⁻²	2 ⁻³	2 ⁻⁴	2 ⁻⁵	2 ⁻⁶	2 ⁻⁷	2 ⁻⁸
0	0	1	0	0	0	1	0	1	1	0	0	0	0	0	0

Figura 4.31: 34,75

27	26	25	24	23	22	21	20	2-12-22-32-42-52-62-72-8								
0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0

Figura 4.32:1,5

27	26	25	24	23	22	21	20	2-12-22-32-42-52-62-72-8								
0	0	1	0	0	1	0	0	0	1	0	0	0	0	0	0	0

Figura 4.33:34,75 + 1,5 = 36,25

Até mesmo o truque de Shift para a direita (dividir por uma potência de dois) e o truque de Shift para a esquerda (multiplicar por uma potência de dois) funcionam...

27	26	25	24	23	22	21	20	2-12-22-32-42-52-62-72-8								
0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0

Figura 4.34:1,5

27	26	25	24	23	22	21	20	2-12-22-32-42-52-62-72-8								
0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0

Figura 4.35:1,5 « 1 = 3

27	26	25	24	23	22	21	20	2-12-22-32-42-52-62-72-8								
0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0

Figura 4.36:1,5 » 1 = 0,75

A notação para ponto fixo éBITS_PARA_PARTE_INTEIRA:BITS_PARA_PARTE_FRACIONÁRIA.
No jogo, a posição do jogador é16:16.A localização da grade está a um turno de distância.
jogador->x»16.

Realizar multiplicações e divisões são casos especiais. Existem duas maneiras de fazer isso:
multiplicar dois valores de 32 bits em 64 bits (e descartar os 16 bits superiores e inferiores) ou

Reduza a precisão de ambos os valores de ponto fixo e multiplique-os por 32 bits. Ambos os métodos envolvem a aceitação de perda de dados por estouro ou subfluxo. Vejamos o exemplo. de $98.7539 * 1.5$.

27	26	25	24	23	22	21	20	2-12	-22	-32	-42	-52	-62	-72	-8
0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	1

Figura 4.37: 98,7539

27	26	25	24	23	22	21	20	2-12	-22	-32	-42	-52	-62	-72	-8
0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0

Figura 4.38: 1,5

Primeiro, desloque ambos os operadores 4 bits para a direita:

27	26	25	24	23	22	21	20	2-12	-22	-32	-42	-52	-62	-72	-8
0	0	0	0	0	1	1	0	0	0	1	0	1	1	0	0

27	26	25	24	23	22	21	20	2-12	-22	-32	-42	-52	-62	-72	-8
0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0

Por fim, multiplique-os:

27	26	25	24	23	22	21	20	2-12	-22	-32	-42	-52	-62	-72	-8
1	0	0	1	0	1	0	0	0	0	1	0	0	0	0	0

Figura 4.39: $98,7539 * 1,5 = 148,125$

$$128 + 16 + 4 + 0.125 = 148.125$$

No exemplo anterior, alguma precisão foi perdida (2-8 pedaços desapareceram durante o $\gg 4$ operação) e um estouro de capacidade poderia ter ocorrido (multiplicar por 3 em vez de 1,5 teria ultrapassado a precisão de 8:8).

No código-fonte, todas as variáveis de ponto fixo têm 32 bits de largura e usam um tipo especial typedef.

```
typedef long fixo;
```

Nem todos os fixos são iguais. Alguns são 16:16, alguns são 24:8. E operações como a multiplicação podem ser implementadas de maneiras diferentes. Isso pode ser bastante complicado para versões modificadas. fixo onde o bit mais à esquerda armazena o sinal:

```
fixo FixedByFrac (fixo um, fixo b) { asm mov si, [WORD
PTR b+2] // sinal do resultado = //
// sinal da fração

asm movimento machado, [PALAVRA] PTR um]
asm movimento cx, [PALAVRA] PTR a+2]

asm ou cx, cx
asm jns aok: // negativo?
asm neg cx
asm neg machado
asm sbb cx, 0
asm xor si, 0x8000 aok: // alternar sinal do resultado

// multiplicar cx:ax por bx
asm movimento bx, [PALAVRA] PTR b]
asm mul bx // fração*fração
asm mov di, dx // di é uma palavra de baixo significado para o resultado //
asm movimento ax, cx
asm mul bx // unidades*fração
asm adicionar machado, di
asm adc dx, 0

// Coloca o resultado dx:ax em complemento de dois
asm teste sim, 0 x8000 O resultado é negativo?
asm jz ansok:
asm neg dx
asm neg machado
asm sbb dx, 0

ansok: // Eu insiro o valor de retorno com ASMs }
```

E muito mais fácil quando o motor conhece dois fixos que têm o mesmo sinal:

```

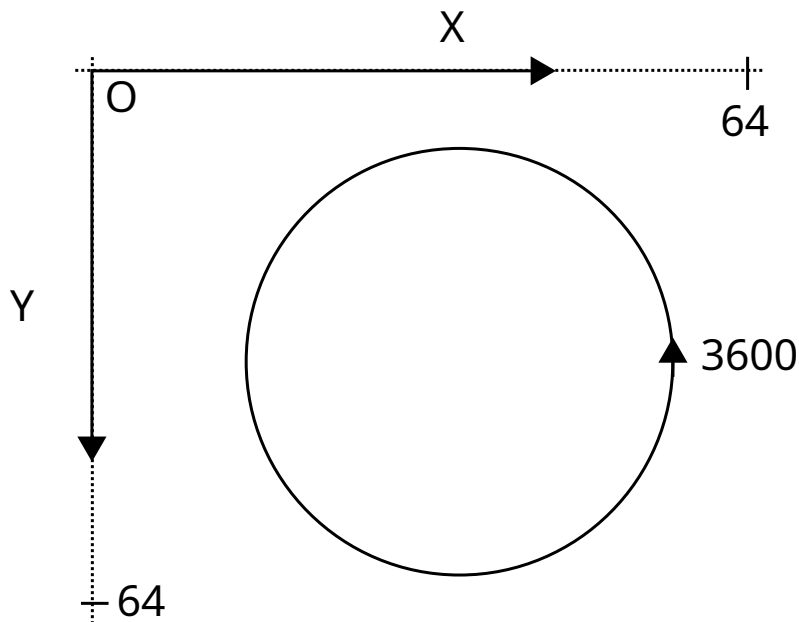
fixo FixedMul (fixo a, fixo b) {
  retornar(a>>8)*(b>>8);
}

```

Curiosidades: O uso da aritmética de ponto fixo não se limitava aos jogos de PC. Muitos consoles de jogos fabricados na década de 90 e posteriormente não possuíam unidades de ponto flutuante para reduzir o custo de produção e maximizar o desempenho do pipeline da CPU. O PlayStation original da Sony (1994) e o Saturn da Sega (1994) são exemplos disso.

4.7.5.2 Sistema de Coordenadas

Com a CPU ponto flutuante/inteiro problema resolvido, é hora de estudar como um raio é projetado. O sistema de coordenadas tem sua origem no canto superior esquerdo. Um mapa é uma grade de 64 por 64 blocos.



Como um bloco tem 8 pés, os mapas podem ter 512 pés de largura e altura. Variáveis de ponto fixo são usadas em todos os lugares. A altura das colunas é 29:3. A posição do jogador é 16:16. Os ângulos, no entanto, são inteiros representando décimos de graus com um intervalo [0, 3600].

4.7.5.3 Mundo Quadrado e Projeção de Raios

Desenhar as paredes consiste em determinar o que é visível, o que não é e o que está na frente do quê. A visão de Michael Abrash sobre o assunto resume tudo:

//

Quero falar sobre o que, na minha opinião, é o problema 3D mais difícil de todos: a determinação da superfície visível (desenhar a superfície correta em cada pixel) e seu parente próximo, o descarte (descartar polígonos não visíveis o mais rápido possível, uma forma de acelerar a determinação da superfície visível). Para maior brevidade, usarei a abreviação VSD para me referir tanto à determinação da superfície visível quanto ao descarte daqui em diante.

Por que acredito que o VSD (Desenho de Superfície Visual) seja o desafio 3D mais difícil? Embora questões de rasterização, como mapeamento de textura, sejam fascinantes e importantes, são tarefas de escopo relativamente limitado e estão sendo transferidas para o hardware à medida que os aceleradores 3D surgem; além disso, elas só escalam com aumentos na resolução da tela, que são relativamente modestos.

Em contraste, a resolução espacial virtual (VSD) é um problema em aberto, e existem dezenas de abordagens em uso atualmente. Mais importante ainda, o desempenho da VSD, quando implementada de forma pouco sofisticada, escala diretamente com a complexidade da cena, que tende a aumentar como uma função quadrática ou cúbica, tornando-se rapidamente o fator limitante na criação de mundos realistas. Prevejo que a VSD se tornará cada vez mais a questão dominante no 3D em tempo real para PC nos próximos anos, à medida que os mundos 3D se tornarem cada vez mais detalhados.

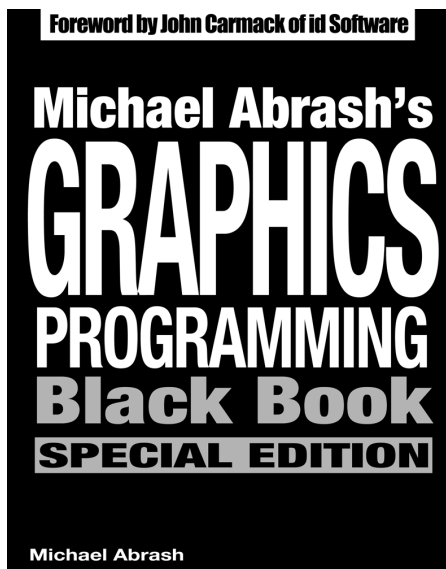
Michael Abrash - Livro Negro da Programação Gráfica

//

Curiosidades: No início dos anos 90, os escritos de Michael Abrash eram uma das raras fontes de informação de alta qualidade no que diz respeito a computação gráfica e programação em linguagem assembly.

Ele publicou dois livros muito elogiados ("Zen of Assembly Language" em 1990 e "Zen of Code Optimization" em 1994), mas foi através de sua coluna "Ramblings in Realtime", publicada mensalmente em [nome da publicação omitido], que ele se consagrou. *Diário do Dr. Dobbs* que ele alcançou notoriedade.

Em 1997, a maior parte do trabalho de Michael Abrash, além de novos artigos sobre o motor gráfico do Quake, foram compilados no livro "The Book of the World". *Livro Negro de Programação Gráfica* O título e as dimensões deste livro são uma homenagem à obra-prima do Sr. Abrash.



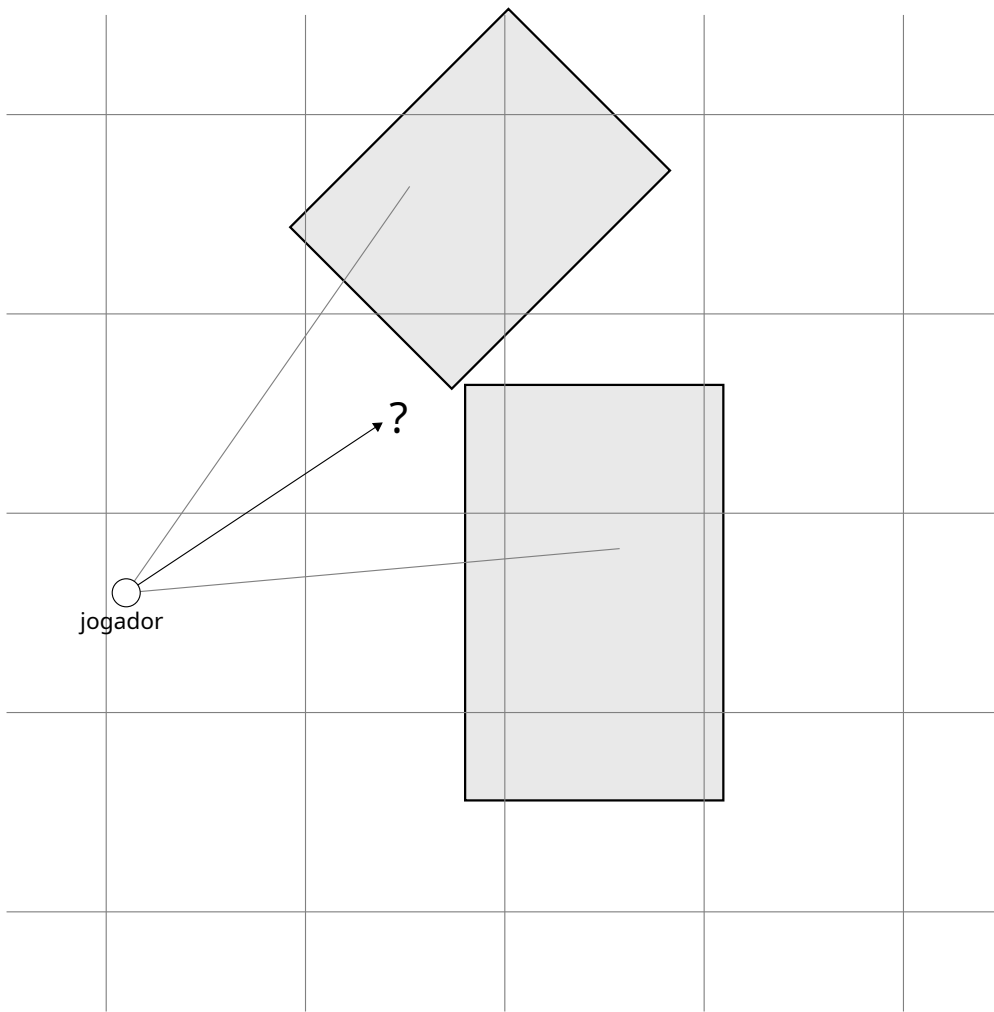


Figure 4.40

VSD é não apenas complicada para fazer certo, é também difícil de ser tão rápido. É fácil de entender a como todos os exemplos aqui que os objetos são frê e como em F nfigura 4.40. Em um mundo sem restrições, é difícil encontrar a intersecção de um raio com um objeto.

Um válido a abordagem seria de iterar sobre todos os objetos no mundo e executar marcha de raios, qual consiste em checar se um raio intersecta os objetos em um intervalo regular. No entanto, isto é muito CPU intensivo e não seria rápido. Chega de consertos de point. Sem mencionar algumas objetos poderia deslizar entre os intervalos, fazendo VSD imprecisões em D.

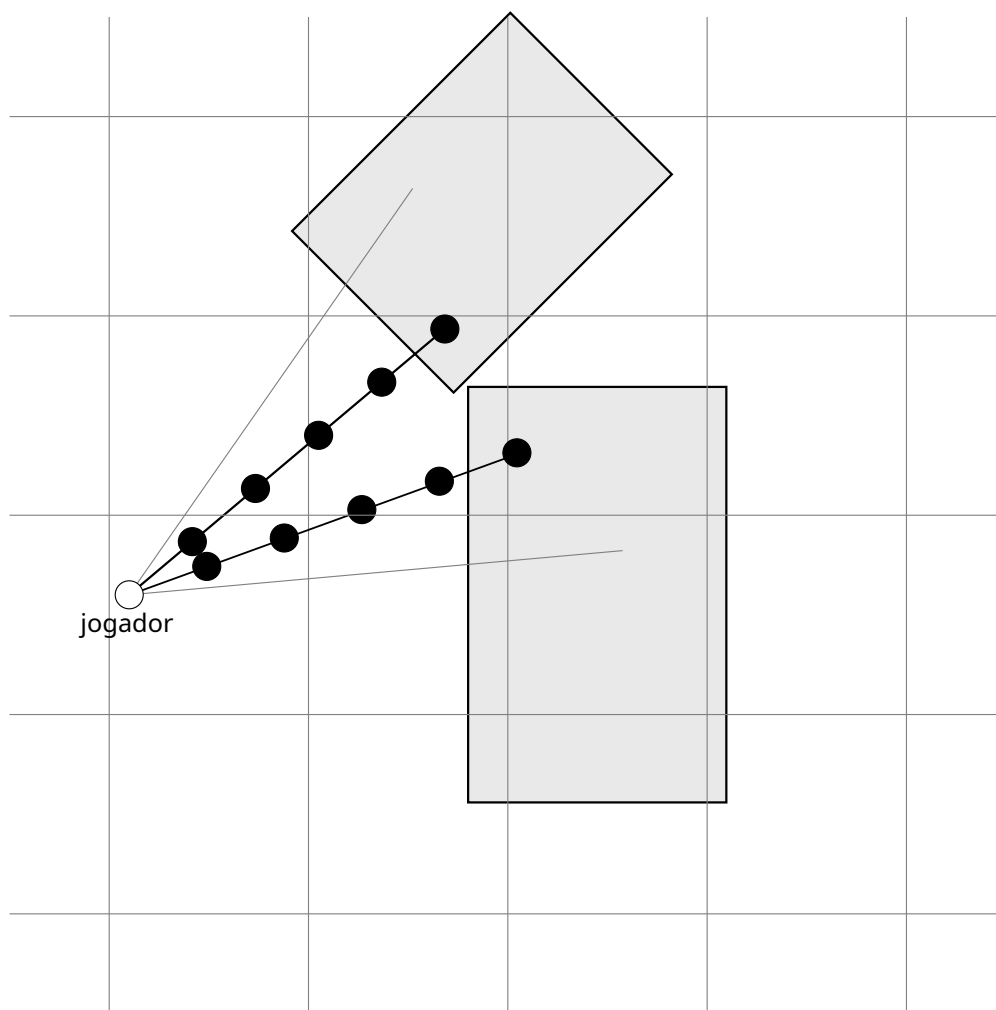


Figura 4.41: Um jogador marchando em direção.

Em um mundo 2D com alguns obstáculos, o problema de encontrar o caminho mais curto torna-se muito mais simples. Se um mapa for feito alinhado a blocos quadrados e apenas distribuído um cruzamento em uma grade, como a solução que produza 100% de e baixo tempo de execução é verificar se o caminho está quando um precisão, o raio cruza a grade.

Isto foi a escolha feita para Wolfenstein 3D, e explique-se por isso que o jogo só pode desenhar perpetuar paredes iculares de 8 pés por 8 pés por 8 feet.

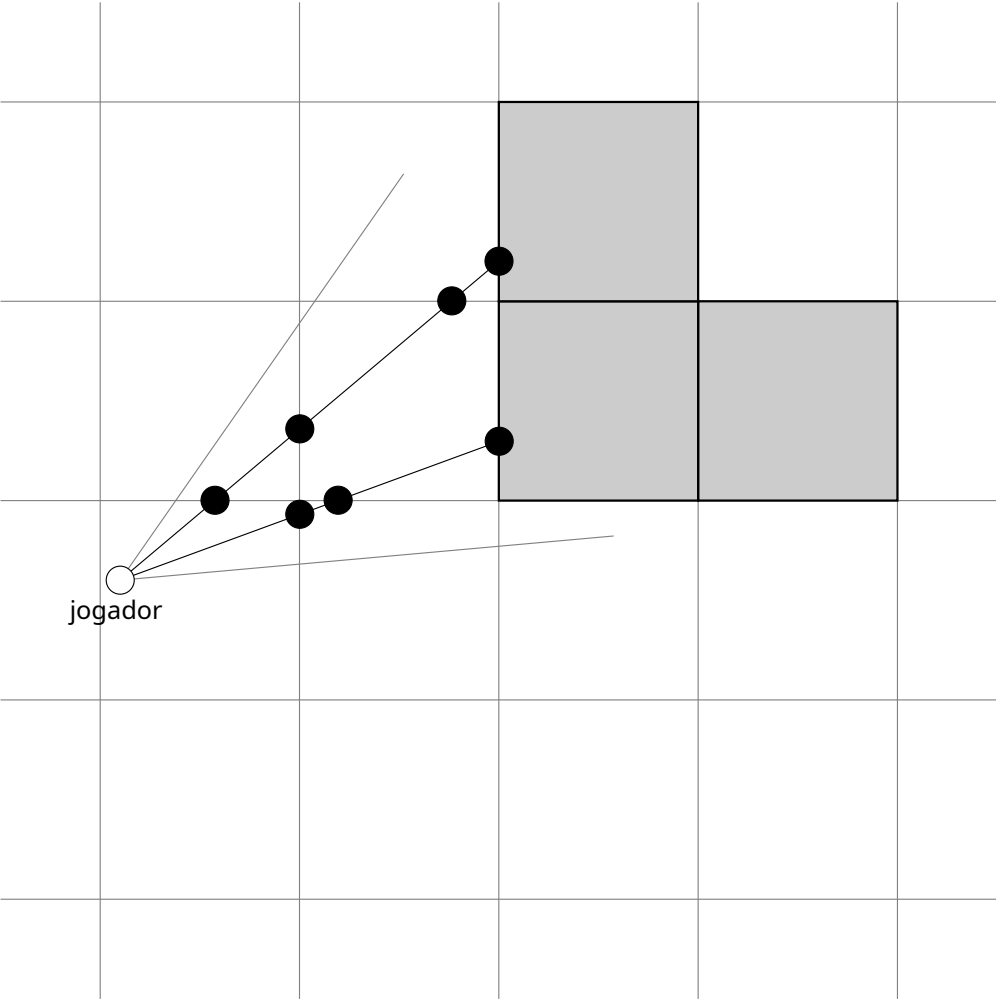


Figura 4.42: Ray casting com um grade.

Esses c várias restrições y limitado o que parentesco d do mundo um desi Gner poderia criar, mas em troca habilitado uso de um rápido a algoritmo.

O Digi Diferencial A n analisador (também conhecido como DA) consideravelmente Não só reduz o número de coordenadas nates para pesquisar para toom para encontrar um C com uma parede. É rapidamente, como também é preciso. sem r(trocadilho intencional) e contato d) para erros.

//

Muito se falou sobre o "ray casting" usado em Wolfenstein, mas o verdadeiro motivo era que eu tinha muitos problemas com a renderização de paredes inteiras no Catacomb 3D. O C3D (e o Hovortank 3D antes dele) vinha com vários glitches gráficos que podiam ocorrer em certas combinações de configurações de blocos do mapa, posição e ângulo de visão. Alguns eram devido a problemas de precisão de ponto fixo que não eram tratados de forma otimizada, e outros eram devido a problemas de clipping e culling que eu só consegui entender completamente alguns anos depois. De qualquer forma, eles me incomodavam bastante. Glitches gráficos espúrios prejudicam muito a imersão em um jogo, e eu queria muito que os jogos da id Software fossem "sólidos como uma rocha".

Havia um custo de desempenho evidente: fazer 320 traçados em um mapa de mosaicos e tratar cada coluna independentemente é muito mais lento do que percorrer alguns segmentos longos de parede. No entanto, o código resultante era pequeno e muito regular em comparação com a complexidade dos meus renderizadores de extensão de parede, e proporcionou a sensação de solidez que eu desejava.

Se você criasse mapas de blocos extremamente irregulares que se transformariam em dezenas de segmentos de parede independentes, o ray tracing poderia começar a parecer uma boa opção em termos de desempenho, mas poucas cenas chegavam perto disso. Essa é exatamente a mesma compensação de desempenho entre ray tracing e rasterização que ainda ocorre hoje, mas agora a questão é "quantas dezenas de milhões de triângulos por quadro são necessárias para que o ray tracing atinja o ponto de equilíbrio" em vez de "quantas dezenas de segmentos de parede".

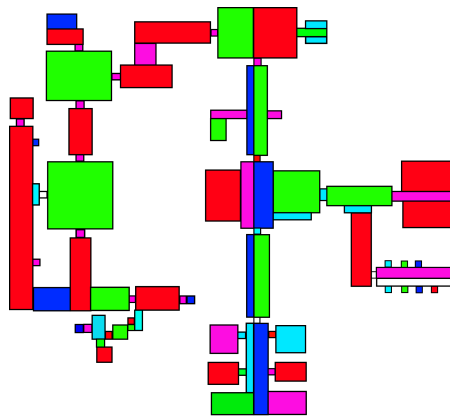
John Carmack - Programador

//

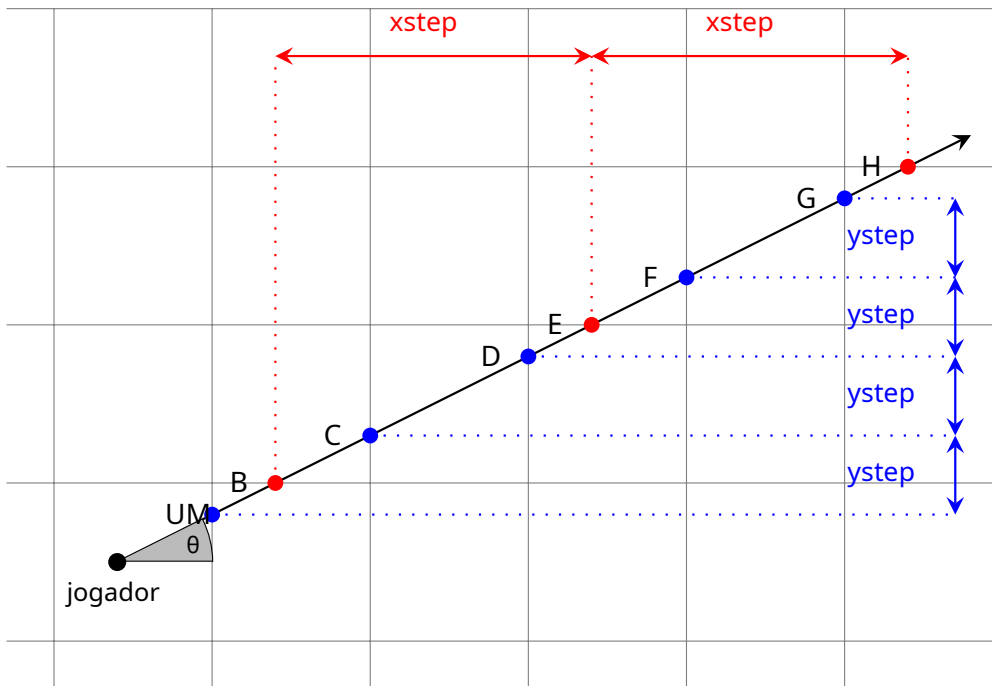
Curiosidades: O custo do raycasting acabaria sendo muito alto para o pequeno processador 5A22 ao portar Wolfenstein 3D para o Super Nintendo.

A primeira versão da adaptação apresentou uma taxa de quadros baixa, o que comprometeu sua viabilidade. John Carmack acabou pré-processando os mapas com Particionamento de Espaço Binário para acelerar a ordenação dos elementos. As paredes foram desenhadas "de perto para longe" com uma matriz de oclusão para evitar sobreposição de elementos.

À direita, o BSP resultante da divisão repetida do mapa. E1M1 em duas, até que cada "área" seja convexa.



E1M1 BSP Partition



O DDA é tão rápido porque, uma vez conhecida a primeira interseção de um raio com um eixo (A para o eixo Y e B para o eixo X), as coordenadas de todas as interseções subsequentes estão a apenas duas adições de distância.

$$C = (X_A + 1, A + 1) \quad D = (X_C + 1, C + 1) \quad F = (X_D + 1, D + 1) \quad G = (X_F + 1, F + 1)$$

Da mesma forma, para as interseções verticais:

$$E = (X_B + 1, B + 1) \quad H = (X_E + 1, E + 1)$$

Observação: queys um d xte psão pesquisas simples nobronzeadomatrix desde = () e re é o xte p= n(90-) quando ângulo do raio em coordenadas do mapa¹⁶.

¹⁶UMi visual revigia de ho peso. Encionamento do círculo trigonométrico pode ser encontrado na página 170.

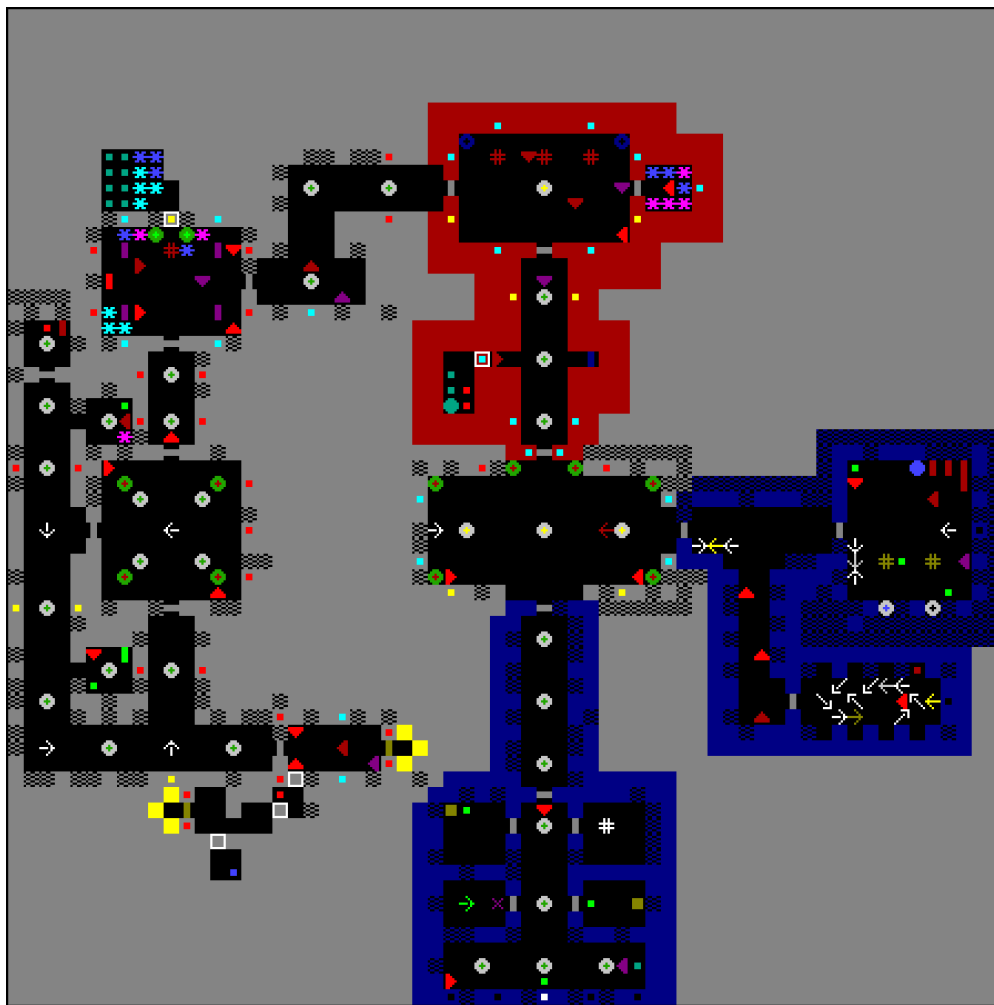


Figura 4.43: O lendário E1M1. O reprodutor é indicado pela seta verde na parte inferior.

4.7.5.4 Chamar Apogee

Como os mapas são rápidos e fáceis de desenhar, foi planejado um concurso. Os jogadores que encontrassem um item especial em um local particularmente difícil de acessar no jogo eram instruídos a ligar para a editora (Apogee).

O labirinto está localizado em E2M8. Atrás de uma floresta de paredes que empurram (quadrados brancos) e chefões furiosos (círculos azuis), o jogador finalmente encontra um sinal estranho (o triângulo vermelho).

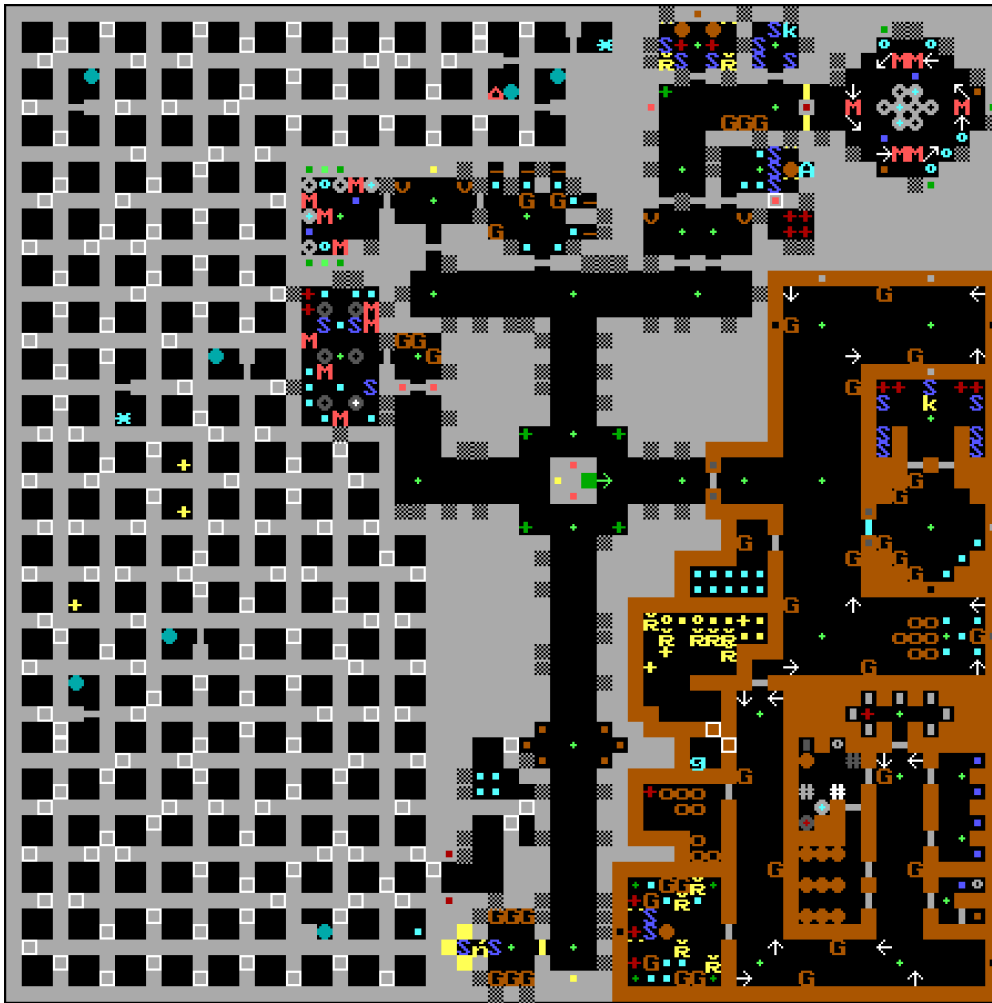


Figura 4.44: Labirinto E2M8.

No entanto, à medida que os jogadores faziam engenharia reversa do formato do mapa e surgiam sites de trapaça, a competição foi cancelada. A placa foi substituída por ossos na segunda versão do GT (1994) e na versão da Activision (1998).

No desenho E2M8, observe que não há paredes de contenção que levem ao triângulo vermelho; a sala está selada.



//

"Ligue para a Apogee e diga Aardwolf." É uma placa que, até hoje, é algo sobre o qual me perguntam muito. Essa placa aparece em uma parede em um labirinto particularmente difícil no Episódio 2, Fase 8 de Wolfenstein 3D. A placa seria o objetivo de um concurso que a Apogee iria promover, mas logo após o lançamento do jogo, uma grande quantidade de programas de trapaça e mapeamento foram disponibilizados. Com esses programas circulando, achamos que seria injusto realizar o concurso e premiar o jogador. A placa acabou ficando no jogo, mas, olhando para trás, provavelmente deveríamos tê-la removido. Até hoje, a Apogee recebe cartas e telefonemas perguntando o que é Aardwolf, frequentemente com a pergunta: "Alguém já viu isso?"

Além disso, em uma questão um tanto relacionada, letras eram exibidas após a maior pontuação na tabela de pontuação em algumas versões do jogo. Essas letras fariam parte de um concurso que foi cancelado antes mesmo de começar, no qual as pessoas ligariam com suas pontuações e nos informariam o código; assim, poderíamos verificar suas pontuações. No entanto, com os programas de trapaça disponíveis, essa ideia também foi descartada.

Basicamente, "Aardwolf" e as letras não significam nada agora.

Joe Siegler - Pioneiros do passado da revolução do shareware

//

Curiosidades: O que é um Aardwolf? Um mamífero com juba listrada (*Proteles cristatus*) do sul e leste da África, semelhante a uma hiena, que se alimenta principalmente de carniça e insetos. Era o mascote da revista *id*, aparecendo nas listas "Tom's Gotta Lists" e na folha de dicas do Commander Keen 6.

//

A razão para a escolha de "aardwolf" foi que esse era o primeiro arquivo de imagem no dicionário NeXT em todas as nossas NeXTStations.

John Carmack - Programador

//

4.7.5.5 Raycasting: Algoritmo DDA

O algoritmo de interseção DDA é implementado em uma rotina de código assembly totalmente desenvolvida manualmente, com 740 linhas: `AsmRefresh`. Aqui está representado em pseudo-C para facilitar a leitura. Consiste em dois laços `while` (um para interseções verticais e outro para horizontais) que se alternam através de comandos `goto`. É altamente heterodoxo e super eficiente.

```

vazioAsmRefresh () {

    para(inteiro i=0 ; i < pixx ; i++) {
        curto ângulo=ângulo médio+ângulo de pixel[pixx]; //
        Configurar os passos x e y com base no ângulo. fazer{

            se(necessário)
                Vá para teste horizontal;

testvertical:

            mover_verticalmente ()
            se (hitdoor)
                HitVertDoor ();
            se (hitwall)
                HitVertWall();
        } enquanto (1);

        continuar;

        fazer{
            se(necessário)
                Vá para teste vertical;

teste horizontal:

            mover_horizontalmente()
            se (hitdoor)
                HitHorizDoor();
            se (hitwall)
                HitHorizWall();

        } enquanto(1);

    }
}

```

Essa implementação resulta em muitos saltos de instruções. Essas instruções destruiriam o cache de instruções e esvaziariam o pipeline em uma CPU moderna. Mas em uma arquitetura com um pipeline pequeno e sem cache de instruções, como o 386, isso não causa a destruição de quadros.

Esta parte do código depende muito dos princípios do círculo trigonométrico. Como precisamos saber quanto avançar verticalmente ao mover-se no eixo X e quanto avançar horizontalmente ao mover-se no eixo Y, obtemos a função é especialmente útil. Isso fica mais fácil de entender com o círculo unitário desenhado (raio do círculo). $r = 1$.

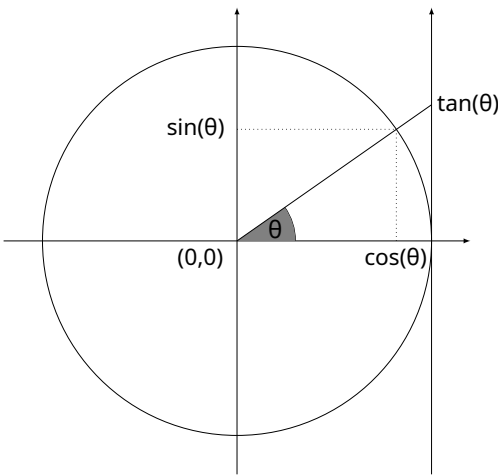


Figura 4.45: Círculo unitário

Ao avançar 1 unidade no eixo X, o raio se move para cima. () no eixo Y. O inverso é calculado da seguinte forma: mova 1 no eixo Y, mova (90 –) no eixo X. Para acelerar cosseno, seno e tangente para os cálculos, o mecanismo usa uma tabela de consulta. Consulte 4.11.1 "Consulta da tabela de cosseno/seno" na página 238.

4.7.5.6 Matemática do Ensino Médio

Antes de prosseguirmos para a próxima seção (que descreve o que é feito com a distância entre o jogador e a parede), aqui está uma breve revisão de um conceito matemático do ensino médio que forma a base da maioria dos cálculos no motor do jogo: SOH-CAH-TOA.

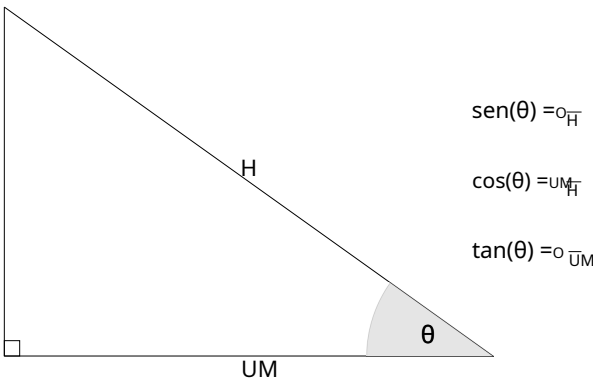


Figura 4. 46: Círculo unitário

"

Não sou nenhum super matemático – aprendi matemática no ensino médio o suficiente para resolver problemas do mundo real com ela.

John Carmack - Programador

Este desenho contém toda a matemática necessária para entender a correção e a coordenação do olho de peixe. As projeções são usadas para posicionar objetos (jogador, inimigos, itens) e calcular a localização dos sons.

4.7.5.7 Cálculo da altura das colunas

Assim que a intersecção entre um raio e uma parede for encontrada a partir da coordenada (viewx, viewy) para (intersecção com o eixo x, intersecção com o eixo y), chegou a hora de calcular a altura da coluna de pixels para este raio. Isso acontece na função CalcAltura.

```
inteiro Altura calculada (vazio)
{
    fixo    gxt, gyt, nx, ny;
    longo   gx, gy;

    gx =  xintercept -viewx;
    gxt =  FixedByFrac(gx,viewcos);

    gy =  yintercept -viewy;
    gyt =  FixedByFrac(gy,viewsin);

    nx = gxt -gyt;

    //
    // calcular proporção de perspectiva (alturanumerador /(nx >>8)) //

    se (nx<mindist)
        nx=mindista;           // Não deixe que a divisão transborde

    asm  movimento machado,[PALAVRA] PTR  numerador de altura]
    asm  movimento dx,[PALAVRA PTR  alturanumerador +2]
    asm  idev      [PALAVRA PTR  nx+1]           // nx >>8
}
```

O código não é o que se esperaria. O algoritmo de raycasting deveria lançar um raio para cada coluna de pixels e usar a distância para inferir a altura da coluna na tela. Portanto, faria sentido ver uma fórmula como:

$$d = \sqrt{dx^2 + dy^2}$$

Em vez disso, o código parece estar fazendo o seguinte:

$$d = \cos(\theta) - \sin(\theta)$$

Tem algo de estranho aqui. Vamos explicar com um exemplo.

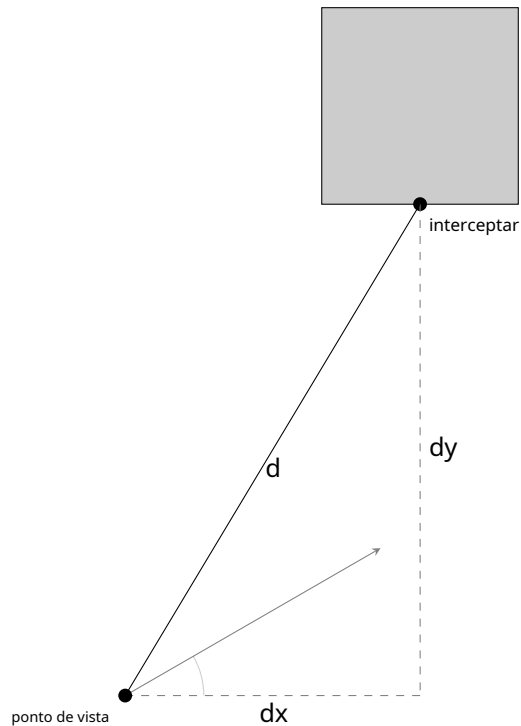


Figura 4.47: Raycasting usando distância d

Neste desenho, o jogador está localizado em ponto de vista com um ângulo de visão. Um raio foi lançado de ponto de vista e bateu de frente com um muro em interceptar. A distância d é uma linha reta entre o jogador/ponto de vista e local onde o raio atingiu a parede. d pode ser obtido com $d = \sqrt{dx^2 + dy^2}$. Repetido para todos os raios, esse algoritmo resultaria em um "efeito olho de peixe".

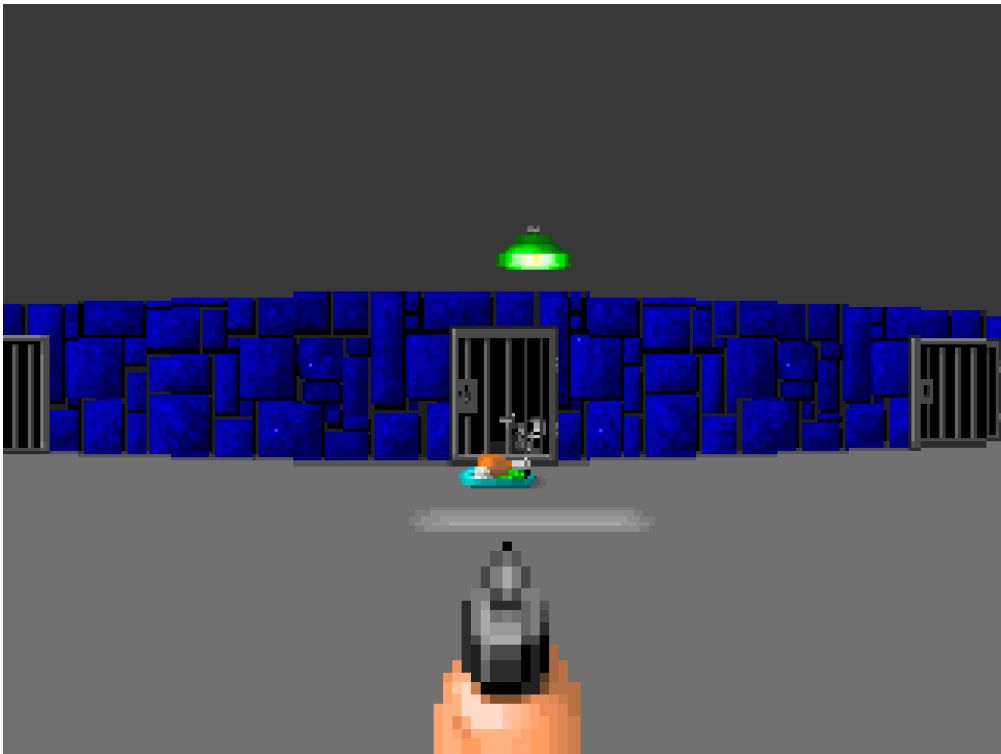
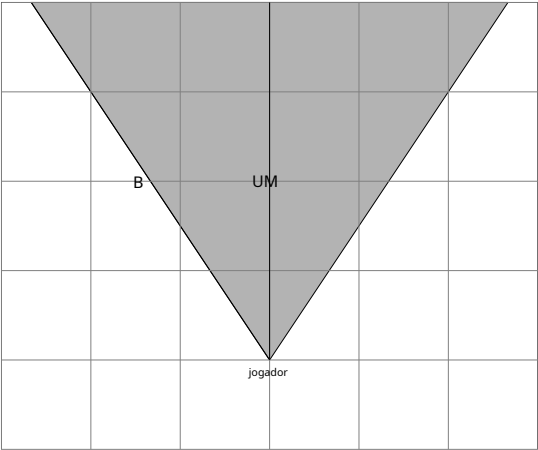


Figura 4.48: Efeito olho de peixe: Leve

Este visual artefato al h um. ontece ser-
causar o distância reta nce do
jogador para a parede é n não constante.
As paredes são mais distante ao lado
do scr e outro antes de representar-
sented s m. alérgeno.

Para demonstrar nestrate o fi sh olho dis-
torção, hesão três capturas de tela
de um mversão modificada são do
motor. É foi alterado t usar direto
distância d em vez de "algo
caso contrário" para c coluna calculada alturas de mn.
No início, frodistância ma e de 32 pés,
distorcer fon é quase nperceptível.



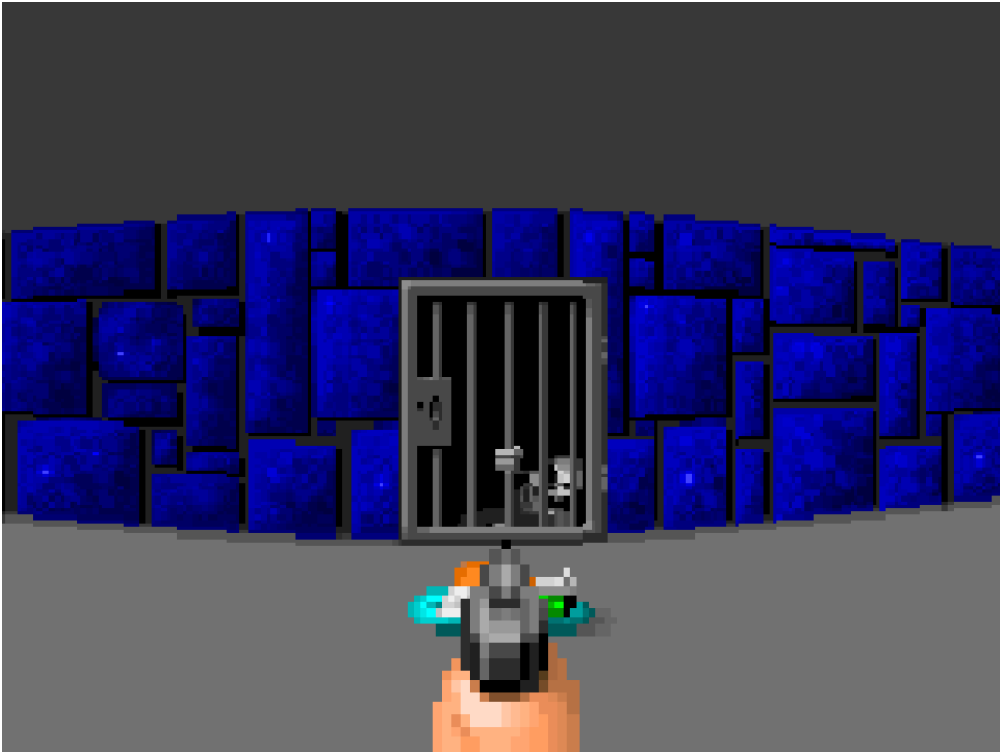


Figura 4. 49:Olho de peixe Efeito: Ruim

De um di sposição de 24 feet, o dis-
torção ca n não deve ser ignorado ed como em
anteriores 32 pés de distância s captura de tela.

Observe que embora to jogador é
ficando cl oser para o c todos, o ra-
tio de A t o B permanece o mesmo.
Apenas o a diferencie completamente ence em pixel
altura wh ena coluna é renderizado
na tela está aumentando.

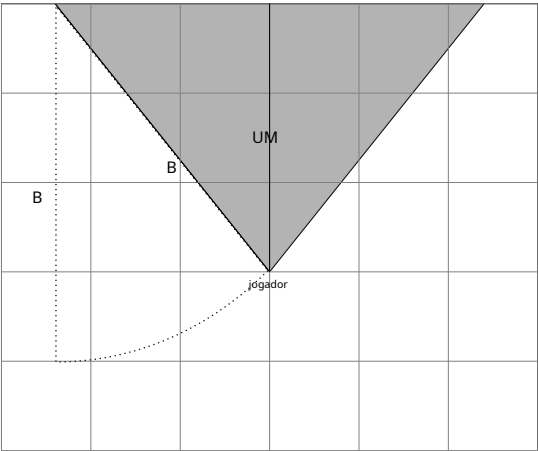
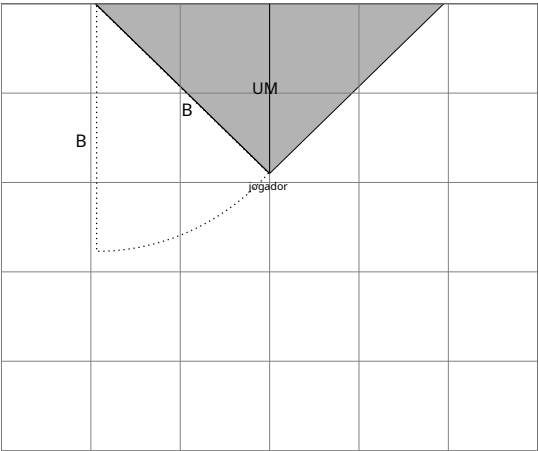




Figura 4.50: Efeito olho de peixe

A 12 pés , a distorção n é reto até o descumprimento. Sformiga com um cl uostrofóbico tocar.

Para evitar th é distorção ue reproduzir um mais plocador ren ditão, o quê deve ser used não é o e dis- direto distânciadb distância ut pprojetado em a câmara um avião (perp) endicular para a vista d direção (z)).



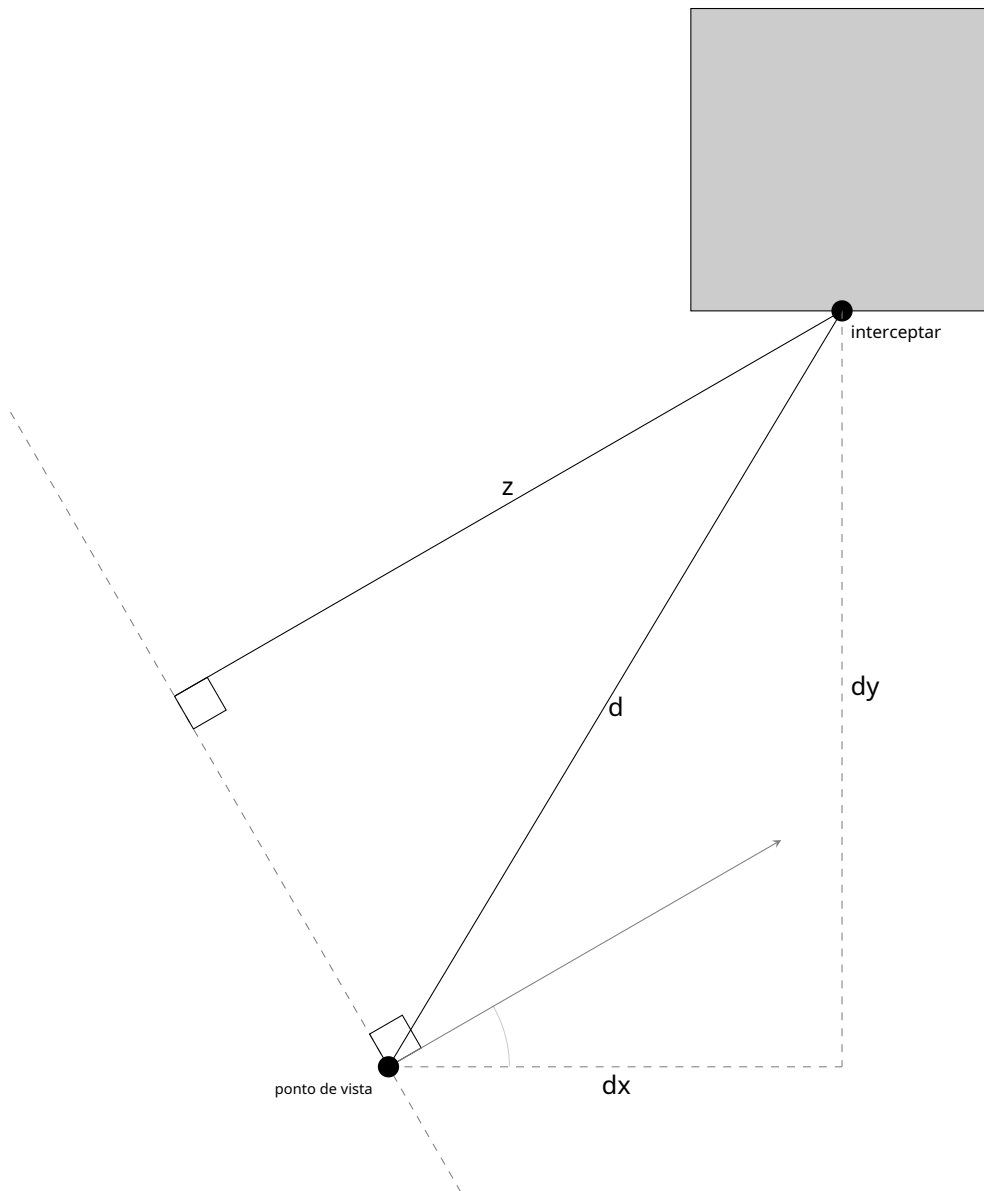


Figura 4.51: distância direta de distância projetada.

Essa projeção (z) é matematicamente difícil de calcular de uma só vez (especialmente com ponto fixo). O truque é dividi-la em dois componentes e usar o mnemônico SOH-CAH-TOA.

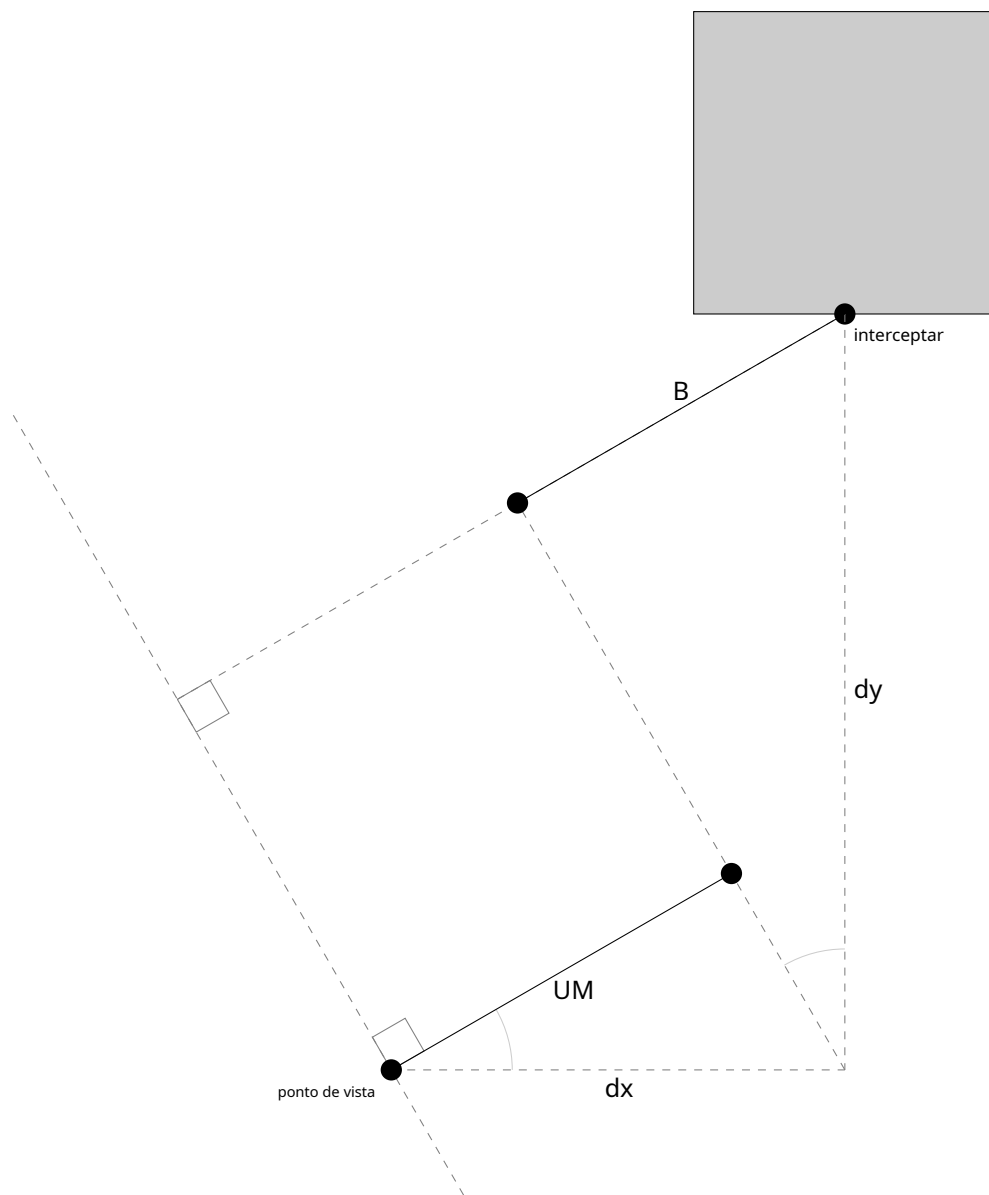


Figura 4.52: Distância projetada é calculado através de dois subcomponentes.

O uso de CAH proporciona $A = \cos(\theta)$ e SOH dá $B = \sin(\theta)$.

Somando-os, obtemos:

$$Z=A+B= \text{*cosseno()} + \text{*pecado()}.$$

Como dx e dy não são distâncias, mas vetores (com sinal), a equação se torna:

$$Z=A+B= \text{*cosseno(-)} + \text{*pecado(-)}$$

que, simplificando, se torna:

$$Z=A+B= \text{*cosseno()} - \text{*pecado()}$$

Para quem gosta de equações, a operação geral pode ser vista como a multiplicação de uma matriz de rotação pelo vetor de intersecção (dx,dy). Isso rotacionaria as coordenadas do espaço do mapa para o espaço do jogador (onde a orientação dos eixos é ditada pelo ângulo de visão do jogador), como mostrado na figura 4.53.

$$\begin{bmatrix} \text{cosseno()} & - \text{pecado()} \\ \text{pecado()} & \text{cosseno()} \end{bmatrix} \begin{bmatrix} dx \\ dy \end{bmatrix} = \begin{bmatrix} \text{*cosseno()} - \text{*pecado()} \\ \text{pecado()} + \text{*cosseno()} \end{bmatrix}$$

Pessoalmente, acho a explicação gráfica com SOH-CAH-TOA mais clara.

Curiosidades: Se você tentou deduzir a fórmula e se confundiu com o sinal do eixo vertical, tente novamente, lembrando que a origem do sistema de coordenadas do mapa 3D de Wolfenstein está no canto superior esquerdo, o que inverte o eixo Y.

4.7.6 Correção do efeito olho de peixe

Essa correção é suficiente para proporcionar linhas retas agradáveis às paredes. Para demonstrar a diferença, as páginas 180 e 181 mostram duas versões da mesma cena lado a lado. Primeiro, sem correção (com lente olho de peixe) e, em seguida, com a distância projetada. Sem vez de distância direta.